SOFTWARE ENGINEERING LAB 3

Final Submission

Multi-Vendor Artisan E-Commerce Marketplace

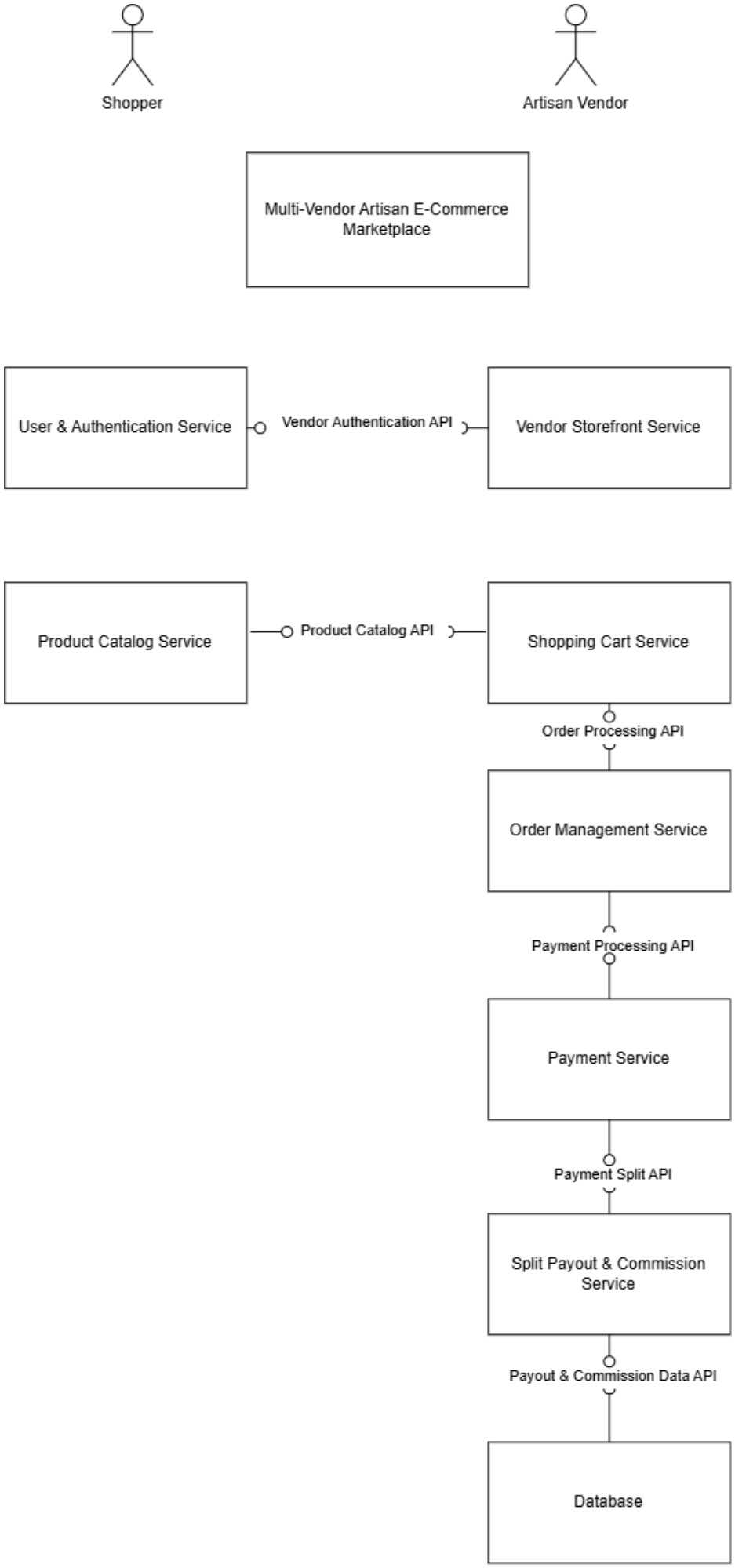
Student Details

Name: Rachit S Mane

SRN: PES1UG24AM214

Selected Architectural Style

Microservices Architecture


Shopper
Artisan Vendor
Multi-Vendor Artisan E-Commerce Marketplace
User & Authentication Service
Vendor Authentication API
Vendor Storefront Service
Product Catalog Service
Product Catalog API
Shopping Cart Service
Order Processing API
Order Management Service
Payment Processing API
Payment Service
Payment Split API
Split Payout & Commission Service
Payout & Commission Data API
Database

SOFTWARE ENGINEERING LAB 3

Architectural Justification

Multi-Vendor Artisan E-Commerce Marketplace

Name	Rachit S Mane	SRN	PES1UG24AM214	Lab	Software Engineering Lab 3

1. Architectural Style Chosen: Microservices Architecture

The Multi-Vendor Artisan E-Commerce Marketplace is structured as a Microservices Architecture. The system is decomposed into eight independently deployable components - User & Authentication Service, Vendor Storefront Service, Product Catalog Service, Shopping Cart Service, Order Management Service, Payment Service, Split Payout & Commission Service and the Database - where each component owns a single business capability and is reachable only through an explicit interface: the Vendor Authentication API, Product Catalog API, Order Processing API, Payment Processing API, Payment Split API and Payout & Commission Data API shown in the component diagram. There is no shared internal state between components; every interaction crosses a published interface.

2. Reason 1: The marketplace decomposes naturally along independent business capabilities

The two actors place largely unrelated demands on the system. An Artisan Vendor creates a storefront and maintains a product catalog, while a Shopper browses products, adds items from several vendors to a cart and checks out. A microservices decomposition allows the vendor-facing Vendor Storefront Service and the shopper-facing Product Catalog and Shopping Cart Services to be developed, deployed and modified independently, because they meet only at the Vendor Authentication API and the Product Catalog API. A change to storefront management therefore requires no redeployment of the checkout path, and a failure in one service is contained behind its interface instead of bringing down the whole marketplace.

3. Reason 2: The split-payment logic must be able to evolve independently of the rest of the system

Splitting a shopper's cart payment among the respective Artisan Vendors and deducting the 5% platform commission is the defining behaviour of this marketplace, and commission and payout rules change far more frequently than browsing or cart behaviour. Isolating that behaviour in a dedicated Split Payout & Commission Service, reached only through the Payment Split API, means the commission rate and the payout calculation can be changed, tested and redeployed on their own. Under a monolithic design the entire marketplace would have to be rebuilt and re-verified for a change confined to one calculation.

4. Security Advantage: Payment and payout data are confined to a single trust boundary

Only the Payment Service and the Split Payout & Commission Service handle payment and payout information, and payout and commission records reach persistent storage solely through the Payout & Commission Data API. No other component in the diagram has a path to that data, so compromising a broadly exposed component such as the Product Catalog Service yields no access to vendor payouts or payment records. In the same way, vendor access to the Vendor Storefront Service is authenticated by the User & Authentication Service across the Vendor Authentication API, which gives one place where vendor identity is enforced rather than authentication logic duplicated in - and independently attackable through - every service.

5. Performance Benefit: Each service scales independently according to its own load

Product browsing generates far more traffic than payment settlement, since a shopper typically views many artisan products before a single checkout. Because the Product Catalog Service is a separate deployable unit, it can be replicated and cached on its own to keep catalog response times fast under peak browsing load, without provisioning matching capacity for the Payment Service or the Split Payout & Commission Service, which serve a much lower request volume. In a single-tier design the entire application would have to be scaled to absorb catalog traffic, consuming resources on components that are not the bottleneck.